

SyBR Documentation

Overview

SyBR offers a fully automated and scalable workflow that integrates synteny analysis, evolutionary breakpoint detection, functional enrichment, and ancestral genome reconstruction to facilitate the study of genome evolution. Even when working with fragmented or incomplete genome assemblies, the pipeline enables the detailed resolution of genomic conservation, chromosomal rearrangements, and the evolutionary context in which these events occurred by integrating high-confidence alignment filtering with downstream analytical modules. Because of this, SyBR is a valuable tool for comparative genomics and for expanding our understanding of the processes that influence chromosome structure and evolutionary history.

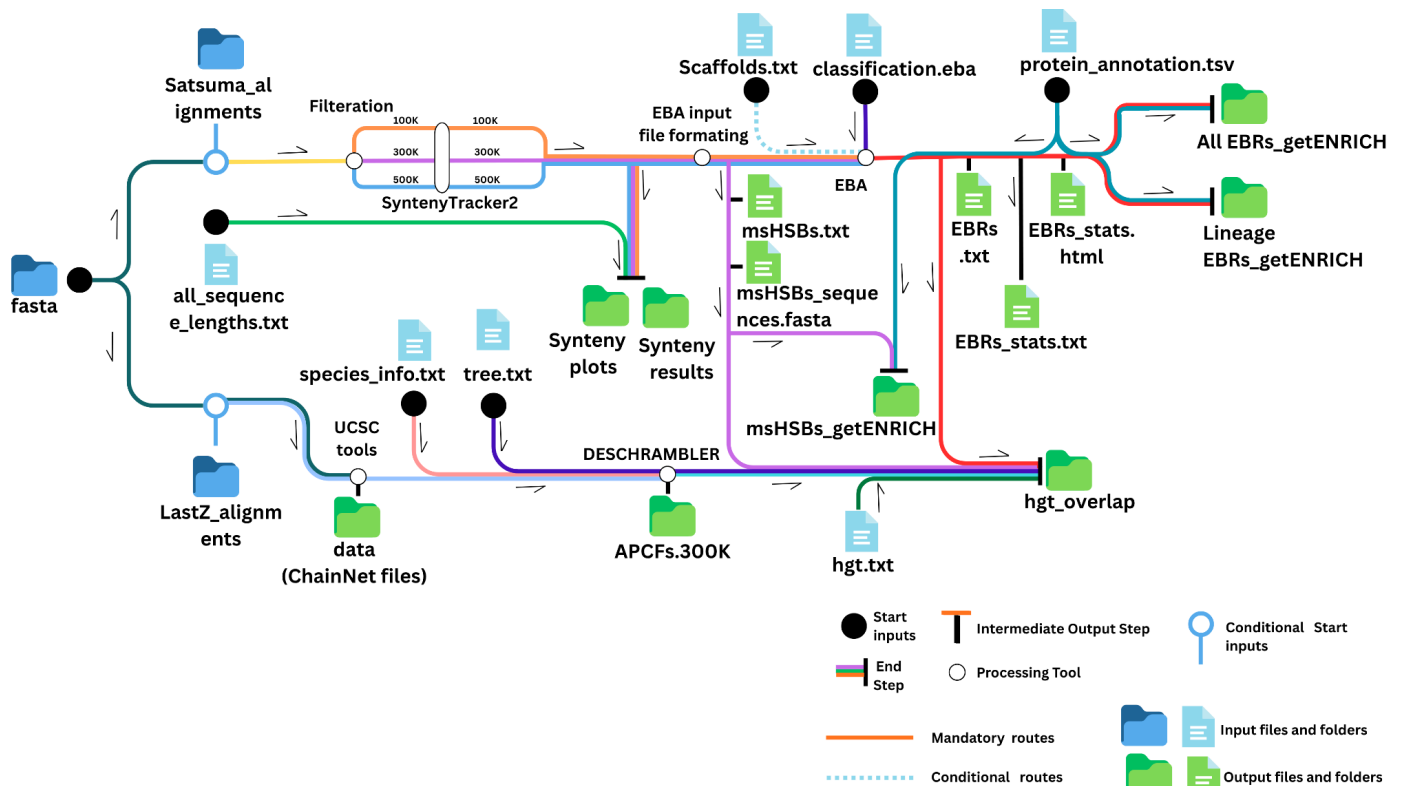


Fig. 1: Schematic illustration of the workflow of the SyBR pipeline. Showing input files/folders (blue in colour) and output files/folders (green in colour). Black-filled dots indicate the input at a particular step. Solid lines with a particular colour show the route of a specific file in the workflow. Dotted lines show the conditional input.

A) Command Line Interface

(1) Installation:

The SyBR pipeline can be installed via GitHub, and dependencies can be installed in a conda environment.

(1.1) Prerequisites:

All the dependencies need to be installed via the conda package manager. So conda should be installed before installing the SyBR pipeline.

(1.1.1) Conda Installation:

This project requires Conda to manage dependencies. If you already have Conda (Miniconda or Anaconda) installed, skip to this.

Check if Conda is Already Installed

Open a terminal and run:

```
conda --version
```

- If you see something like **conda 24.x.x** — Conda is already installed, skip the section below.
- If you see '**command not found**,' install Conda.

If Conda is not installed on your system, follow this step-by-step guide for Ubuntu:



https://medium.com/@mustafa_kamal/a-step-by-step-guide-to-installing-conda-in-ubuntu-and-creating-an-environment-d4e49a73fc46

(1.2) Installation of SyBR:

(1.2.1) Clone the Repository:

Clone the project from GitHub using the following command:

```
git clone https://github.com/BioinformaticsOnLine/Sybr.git
```

(1.2.2) Set File Permissions:

Grant the necessary permissions to all files and directories:

```
chmod -R 777 Sybr
```

(1.2.3) Navigate to Project Directory:

Change the directory into the project folder

```
cd Sybr
```

(1.2.4) Create Conda Environment:

Install all required dependencies by creating the Conda environment from the provided YAML file

```
conda env create -f install_sybr_dependence.yml
```

(1.2.5) Activate Conda Environment:

```
conda activate sybr
```

(1.3) Preparation of Input Files:

For the user's reference, we provide an example of input data. User can download the example data from these links:

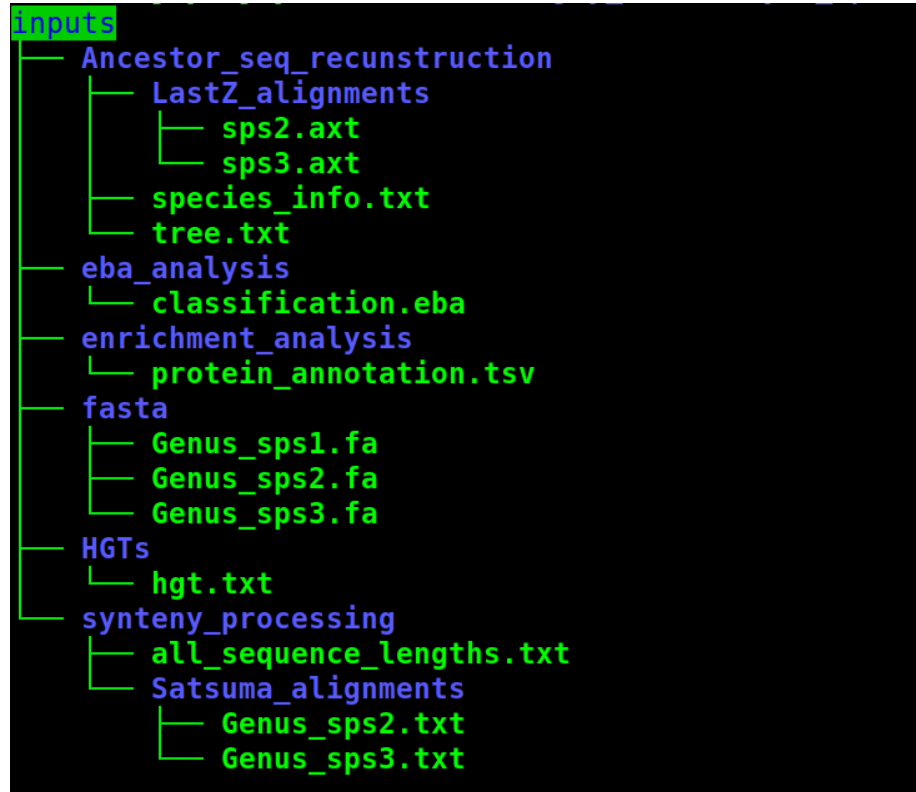
Fast track: - Example input files with pre-computed alignments:

<https://doi.org/10.6084/m9.figshare.32315682>

Slow track: -Example input files without pre-computed alignments:

<https://doi.org/10.6084/m9.figshare.32315892>

For each module, there are separate input files and folders. Users can provide the inputs according to the result requirements, but the file structure is fixed. The user should provide the input in the same structure as this:



This pipeline is run in eight steps. Users can select the modules to run based on their needs. The steps are as follows:

```
# — Pipeline stages to run —
run_stages:
  run_satsuma_alignment: false
  run_lastz_alignment: false
  syntenic_processing: true
  eba_analysis: true
  enrichment_analysis: true
  chainNet_generation: true
  Ancestor_seq_reconstruction: true
  hgt_overlap_analysis: true
```

To activate the module to run, users need to set it to **true** in **run_sybr_config.yaml**. And to deactivate the module, set it to **false**. For every module, specific file formats and structures are required. Explained in detail below:

(1.3.1) Common input files for multiple modules:

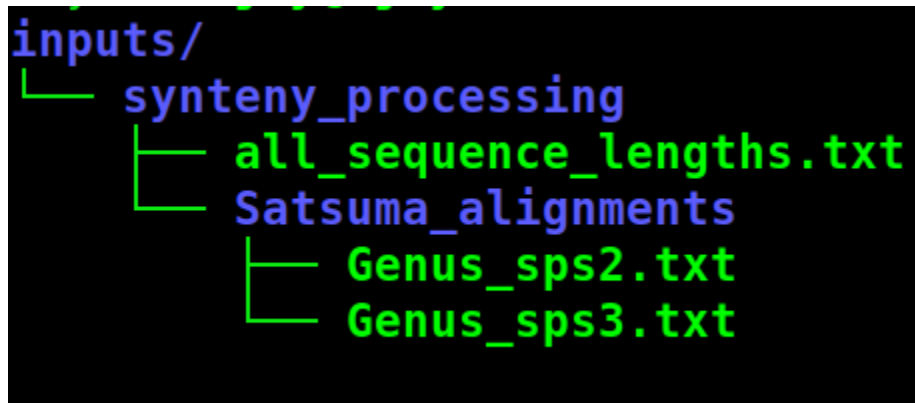
(1.3.1.1) fasta:

In the input folder, folder named "fasta" contains the input fasta files which are common for the modules **run_satsuma_alignment**, **run_lastz_alignment**, **eba_analysis**, and **chainNet_generation**. The format of input fasta files should be like: **Genus_species.fa** and every header should contain numeric value only.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr$ cd inputs/
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs$ ls
Ancestor_seq_reconstruction  eba_analysis  enrichment_analysis  fasta  HGTs  syntenic_processing
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs$ cd fasta/
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/fasta$ ls
Genus_sps1.fa  Genus_sps2.fa  Genus_sps3.fa
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/fasta$ head Genus_sps1.fa
>1
TGATAATCTATTATTTTCCAATGTTGgattataaacaagttaacatgaaatcaaattt
agcCATTTTCATATGATTGTTCTAATGATACTTCTTCTaattttgtattattcatcGATAA
GTAAGTTTCACTTTCGGACAAAGTGATAAAATCGAACATTGTTTCCAAAAAGCTAAGATT
AGATTGATTGAGATGAAAGTCACTGAGAATATGTTGAAGTGGATAATGAGAATGTTCAAG
TATTGATAAACACATCTCTCTTACAGAGTGAACAACTTCATCAAATGACCAATCTCCATT
GAGCTGAACACGATATGGTAATGTTGCCACAAACATTCTATCATATTATACAATTCAGA
TCGATATCGATTGGCATTATAGTACCAACACACAAATCAGTTTGATTATGagataattt
gaatagaaatcacatAAAAATATCGATAGTCCAAGTTGAAACAGTGAAAGATGATGTGATGA
TGCAATAATTTAGAAATGATGTACATATAGCATCATCTAATGTTATTGAGTTCTAGAAGC
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/fasta$
```

(1.3.2.1) Input Files for synteny_processing:

The file structure of **synteny_processing** should be like this:



(1.3.2.2) all_sequence_lengths.txt:

This is a tab-delimited text file with three columns and without headers. The first column is the **chromosome** or **scaffold** number; this number should be **numeric** only (e.g., 1, 2, 3...). The second column contains the size or length of the **chromosome** or **scaffold**. The third column is for the name of the **species**. It should match what is mentioned in the other input files. This file contains information on all sequences from all species. The lengths of the reference genome's sequences should be written three times, once for each resolution selected for synteny analysis.

In the SyBR tool's home folder, the **genome_length_maker.sh** script is provided, which creates **all_sequence_lengths.txt**. The usage of this script is like this:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr$ ./genome_length_maker.sh -h
Usage: genome_length_maker.sh -r <reference_species> [options]

Required:
  -r, --reference Basename of the reference FASTA (no extension)

Optional:
  -i, --input Folder containing all FASTA files (default: current directory)
  -o, --output Output file path (default: <input_folder>/all_sequence_lengths.txt)
  -k, --resolutions Comma- or space-separated resolutions in kb (default: 100,300,500)
  -h, --help Show this help

Examples:
  genome_length_maker.sh -r Genus_sps1
  genome_length_maker.sh -r Genus_sps1 -i inputs/fast
  genome_length_maker.sh -r Genus_sps1 -i inputs/fast -o inputs/synteny_processing/all_sequence_lengths.txt
  genome_length_maker.sh -r Genus_sps1 -i inputs/fast -k 100,300,500,1000
  genome_length_maker.sh -r Genus_sps1 -i inputs/fast -o my_lengths.txt -k 200 400
(sybr) ajay@ajay-SS400TR-34:~/Sybr$
```

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/synteny_processing$ ls
all_sequence_lengths.txt  Satsuma_alignments
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/synteny_processing$ cat all_sequence_lengths.txt
1      18146847      Genus_sps1:30k
3      20354754      Genus_sps1:30k
5      16930519      Genus_sps1:30k
1      18146847      Genus_sps1:60k
3      20354754      Genus_sps1:60k
5      16930519      Genus_sps1:60k
1      18146847      Genus_sps1:90k
3      20354754      Genus_sps1:90k
5      16930519      Genus_sps1:90k
1      20317728      Genus_sps2
3      22412911      Genus_sps2
5      23051187      Genus_sps2
1      20932626      Genus_sps3
3      21077837      Genus_sps3
5      17364795      Genus_sps3
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/synteny_processing$
```

This script calculates the lengths of all sequences across all genomes in the folder where it runs. The name of all the genomes should be written as **Genus_species.fasta**.

(1.3.2.3) Scaffolds.txt:

This file contains the names of species whose genome assemblies are **not at the chromosome level**. If all the genome assemblies are chromosome-level, then modify the **pipeline_paths.yaml** from this

```
scaffolds_file:      "inputs/synteny_processing/Scaffolds.txt"
```

To this:

```
scaffolds_file:      "ALL_CHROMOSOMES"
```

(1.3.2.4) Satsuma_alignments:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/synteny_processing/Satsuma_alignments$ ls
Genus_sps2.txt  Genus_sps3.txt
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/synteny_processing/Satsuma_alignments$ head Genus_sps2.txt
1      60754      61163      3      694776      695185      0.562347      +
1      61245      61679      3      695291      695726      0.571429      +
1      137760      137883      3      4500431      4500554      0.528455      +
1      137897      139971      3      4500616      4502696      0.828833      +
1      158055      158342      3      80801      81088      0.710801      +
1      158813      159447      3      81141      81777      0.701893      +
1      223892      224179      3      47888      48175      0.609756      +
1      224208      224421      3      48148      48361      0.586854      +
1      224259      224418      3      48148      48307      0.503145      +
1      224419      224597      3      48345      48523      0.595506      +
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/synteny_processing/Satsuma_alignments$
```

In the **Satsuma_alignments** folder, all the alignment files present are aligned by the Satsuma tool. The format of the name of alignment files should be **Genus_species.txt**. Each file is a tab-delimited text file with 8 columns; it should not have headers. The first column is the **chromosome number / scaffold number** (numeric only), the second column is the **reference start coordinates**, the third column is the **reference end coordinates**, and the fourth column is the **query chromosome/scaffold number (numeric)**. The fifth column is **query start coordinates**, the sixth column is **query end coordinates**, the seventh column is the **score** of Satsuma, and the eighth column is the strand.

(1.3.3) Input Files for eba_analysis:

The file structure to run **eba_analysis** should be like this:

```
./inputs/  
└─ eba_analysis  
    └─ classification.eba
```

(1.3.3.1) classification.eba:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/eba_analysis$ ls  
classification.eba  
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/eba_analysis$ cat classification.eba  
lineage=  
Genus_sps1=Genus_sps2,Genus_sps3  
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/eba_analysis$
```

This is a **classification** file for **EBA**. In this file, the user can classify groups of related species by **node** or **order**. The first line of this file should be **lineage=**, the second line should start with the name of the reference species, and all remaining species names should follow the = sign. In the next lines, species can be classified into different **groups** like shown in the picture below (**this file is not used in the example**).

```
lineage=  
Genus_sps1=Genus_sps2,Genus_sps3,Genus_sps4,Genus_sps5  
node1=Genus_sps2,Genus_sps3
```

For a detailed explanation, the user should visit the documentation of the EBA tool:

https://pure.aber.ac.uk/ws/portalfiles/portal/10624966/Narayan_J.pdf

(1.3.4) Input Files for enrichment_analysis:

The file structure for the **enrichment_analysis** module should be like this:

```
./inputs/  
└─ enrichment_analysis  
    └─ protein_annotation.tsv
```

(1.3.4.1) protein_annotation.tsv:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/enrichment_analysis$ ls  
protein_annotation.tsv  
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/enrichment_analysis$ head protein_annotation.tsv  
CP075492.1    4391    5403    3    UJR06537.1  
CP075492.1    8205    9286    3    UJR06538.1  
CP075492.1   13149   16010    3    UJR06539.1  
CP075492.1   19366   33899    3    UJR06540.1  
CP075492.1   45201   48426    3    UJR06541.1  
CP075492.1   49650   50896    3    UJR06542.1  
CP075492.1   51562   53479    3    UJR06543.1  
CP075492.1   60237   61256    3    UJR06544.1  
CP075492.1   62461   64604    3    UJR06545.1  
CP075492.1   66488   70489    3    UJR06546.1  
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/enrichment_analysis$
```

This is a **tab-delimited** text file **without headers**, with **five columns**. The first column contains the **chromosome accession number** of the reference genome. The second column represents the **start coordinates** of the gene in the reference genome. The third column is the **end coordinates**. The fourth column contains the **chromosome number (numeric only)**, and the fifth column contains the accession number for **protein-coding genes or gene ID according to the need in the getENRICH tool**.

Users can download this file from NCBI.

Go to NCBI > Search your organism > Genbank annotation > Download .tsv format

(1.3.5) Input Files for Ancestor_seq_reconstruction:

The file structure to run **Ancestor_seq_reconstruction** should be like this:

```
./inputs/  
└─ Ancestor_seq_reconstruction  
    └─ LastZ_alignments  
        └─ sps2.axt  
        └─ sps3.axt  
    └─ species_info.txt  
    └─ tree.txt
```

(1.3.5.1) LastZ_alignments:

In the **inputs** folder **LastZ_alignments**, the user should put lastz alignment files with the **.axt** extension. The file name format should be: **species.axt**.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction/LastZ_alignments$ ls
sps2.axt  sps3.axt
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction/LastZ_alignments$ head sps2.axt
0 chr1 5664 5903 chr3 598268 598499 + 11807
TCTCTCATTATATGCTACTTACAGTACTTCTCCCTAATAACGACActaaaatgaaactttattttgtttttatttcgcGTTTCATCTATTTACGACCGATTTCAAACAACCTTTATATTTGGGGAATCTTTGGATGA
CGCTCTTTGGAATGATATATTTTC-CGATCCATTTTCTCATATGTTTTCATGGAGATATCACCGGTCTACGATTGTGATGCTTGGTGTCGTTATAACGTTCTACT
tttttcagtat-ttcacatgaaactgcgcaccca-----CACTAAAGTGAAACTTCATTTT-GTTTCTATCGGTGTTCTCTATTTACGTCGAATTTCTAATAACTTTATATTTGGGGAATCCTTGGATAA
CGCTCTTTGGAATGATGTTTCTGCCCCCCTTTATCATTATTATTTTCATGGAGAAATCACCGGTCTACGATGAAAGTGACTTGCTGCTTACTGGGTACCACt

1 chr1 5683 5933 chr3 9780111 9780362 + 13496
TACAGTACTTCTCCCTAATAACGACActaaaatgaaactttattttgtttttatttcgcGTTTCATCTATTTACGACCGATTTCAAACAACCTTTATATTTGGGGAA-TCTTTGGATGACGCTCTTTCGAATGATAT
ATTTC-CGATCCATTTTCTCATATGTTTTCATGGAGATATCACCGGTCTACGATTGTGATGCTTGGTGTCGTTATAACGTTCTACTTCGTAAT-AGAGACACTCAGCTATTTTTC
TACAGTACTTTTACCTAATAACGACACTAAAGTGAAACTTCATTTT-GTTTCTATCGGTGTTCTGCTATTTTACATCCAATTTCCAATAAATTTATATTTGGGGAATCCTTGGATGACGCTCTTTCGAATGATGT
GTTTCTCGACCCA-TTTATCATTGTTTTCATGGAGAAATCATCGGTCTACGATGAAAGTGCTTGGTGCTGTTATTAGGGAATACTGTAATAAAAAATACAATTATACTTTT

2 chr1 5683 5909 chr3 22190857 22191071 + 15714
TACAGTACTTCTCCCTAATAACGACActaaaatgaaactttattttgtttttatttcgcGTTTCATCTATTTACGACCGATTTCAAACAACCTTTATATTTGGGGAATCTTTGGATGACGCTCTTTCGAATGATATA
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction/LastZ_alignments$
```

(1.3.5.2) species_info.txt:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction$ ls
LastZ_alignments  species_info.txt  tree.txt
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction$ cat species_info.txt
sps1 0 1
sps2 1 1
sps3 2 1
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction$
```

This file contains information about the **reference**, **descendant**, **outgroup**, **chromosome-scale**, and **scaffold-level assembly**. There are three columns in this file separated by a **space**. The first column is the **species name**, the same as the file's name in the **seq** and **LastZ_alignments** folders. The second column represents the **species' role**:- 0 = **reference** , 1 = **descendant**, 2 = **outgroup**. The third column represents the **assembly level**:- 1 = **chromosome-scale**, 0 = **scaffold-level**.

Please refer to <https://github.com/jkimlab/DESCHRAMBLER> for details.

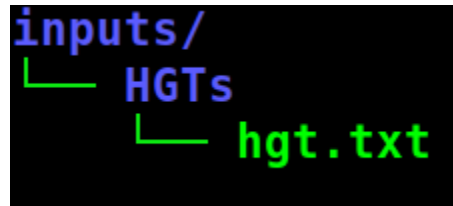
(1.3.5.3) tree.txt:

This file contains the Newick tree for the species listed in the species_info.txt file. The target ancestor must be specified by the "@" symbol. Please refer to <https://github.com/jkimlab/DESCHRAMBLER> for details.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction$ ls
LastZ_alignments  species_info.txt  tree.txt
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction$ cat tree.txt
((sps1:0.02,sps2:0.02):0.02,sps3:0.02);
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/Ancestor_seq_reconstruction$
```

(1.3.6) Input Files for hgt_overlap_analysis:

The file structure to run **hgt_overlap_analysis** should be like this:



(1.3.6.1) hgt.txt:

This input file contains four columns separated by a tab. This file belongs to the information of HGTs. The first column is the chromosome number, the second column is the start coordinates, the third column is the end coordinates, and the fourth column contains the HGTs gene.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/HGTs$ head hgt.txt  
chr1    6521    19748556    NM_670488  
chr1    1930278 2079661    NM_240063  
chr1    11228350    11875314    XM_942651  
chr1    30961815    32801597    XM_187242  
chr1    37133145    38202199    XR_095326  
chr1    39555029    39606752    NM_256737  
chr1    44643799    45496611    XR_504741  
chr1    51058815    52183987    XR_146992  
chr1    55521684    57268883    XR_408397  
chr1    57797848    59399511    XM_107176  
(sybr) ajay@ajay-SS400TR-34:~/Sybr/inputs/HGTs$
```

(2) How to run the SyBR pipeline:

After arranging the inputs folder, the user needs to set **run_sybr_config.yaml** and **pipeline_paths.yaml** files.

(2.1) run_sybr_config.yaml:

After setting the file structure in the input folder, the user must create a config file named **run_sybr_config.yaml**. Provide the path of the input folder in **base_input_dir** and the path of the output folder in **base_output_dir**. After setting the paths, set the modules you want to run to true. In **reference_name** and 'r:' in **eba**, provide the name of the **reference species**.

```

# run_sybr_config.yaml - User-facing config
# Edit this file to control pipeline behaviour
#
# — Base I/O directories —
# Set these to absolute paths if your inputs/outputs live outside the
# workflow directory. Leave them as "inputs" / "outputs" to use the
# default folders relative to the workflow root.
base_input_dir: "/home/ajay/Sybr/inputs"
base_output_dir: "/home/ajay/Sybr/outputs"
# — Pipeline stages to run —
run_stages:
  run_satsuma_alignment: false
  run_lastz_alignment: false
  syntenic_processing: true
  eba_analysis: true
  enrichment_analysis: true
  chainNet_generation: true
  Ancestor_seq_reconstruction: true
  hgt_overlap_analysis: true
# — Species / reference names —
reference_name: "Genus_sps1"
reference_species: "sps1"
# — EBA parameters (user-facing) —
eba:
  n: 2          # number of EBA iterations
  r: "Genus_sps1" # reference species name
  p: 60         # resolution parameter # M

```

For **reference_species**, provide the **species name**. In **EBA**, ‘**n:**’ represents the number of species **excluding the reference species**, while ‘**p:**’ denotes the primary resolution for syntenic processing.

(2.2) pipeline_paths.yaml:

This config file remains unchanged. The user needs to change this configuration file if all the genomes in the analysis are chromosome-level. In this case, the value of **scaffolds_file** will be **ALL_CHROMOSOMES** instead of the path to Scaffolds.txt. **If some genomes are not chromosome-level, then leave them.**

```

# — EBA analysis - paths —
eba_format:
  pre_EBA_dir: "outputs/eba_analysis/pre-EBA"
  scaffolds_file: "ALL_CHROMOSOMES" # use path "inputs/syntenic_processing/Scaffolds.txt" or "ALL_CHROMOSOMES" to include everything
  eba_input_dir: "inputs/eba_analysis/EBA-input"
  mshsbs_dir: "outputs/eba_analysis/mshsbs"
  ebrs_dir: "outputs/eba_analysis/EBRs"
  copy_destination_dir: "outputs/eba_analysis/EBA_results"

```

(2.3) sybr.sh:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr$ ./sybr.sh -h

SYBR
=====

=====
    Snakemake Pipeline
=====

Usage: ./sybr.sh [OPTIONS]

Options:
  -c, --config FILE      Configuration file (default: run_sybr_config.yaml)
  -P, --paths FILE       Static paths file (default: pipeline_paths.yaml)
  -j, --cores N          Number of cores (default: all available)
  -t, --target RULE      Target rule (default: all)
  -l, --log FILE         Log output to file
  -u, --unlock           Unlock working directory
  -n, --dry-run          Dry run (simulate pipeline)
  -k, --keep-going       Keep going on independent job failures
  -v, --verbose          Verbose Snakemake output
  -s, --skip-validation  Skip input validation
  -C, --clean            Remove intermediate files after successful completion
  -w, --window-sizes     Comma-separated window sizes in bp (e.g., 100000,300000,500000)
  -p, --step-size        Step size in bp for syntenic assignment (default: 30000)
  -h, --help            Show this help

Examples:
  ./sybr.sh -c config.yaml -j 8                # Run with default settings
  ./sybr.sh --window-sizes 200000,400000 --step-size 50000 # Custom window sizes and step size
  ./sybr.sh --window-sizes 100000                # Single window size
  ./sybr.sh --step-size 25000                    # Custom step size with default windows
  ./sybr.sh --clean                             # Delete intermediate files after run

Note: Window sizes and step size only affect syntenic assign rules. If not specified,
      defaults are: window sizes = 100000,300000,500000 and step size = 30000.
```

Sybr.sh is a helper script for running this Snakemake pipeline. To see all the options in the pipeline, run this script with the help option: **sybr.sh -h**.

To run the example dataset, user can use the script **example_run.sh**. The command used in this script is:

```
./sybr.sh -j 20 -w 30000,60000,90000 -p 3000 -u -C
```

The very basic usage of SyBR pipeline is this:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr$ ./sybr.sh -j 8 -C
```

In this command, the **window size** will default to **100000**, **300000**, and **500000**. And the **step size** will be **30000**. To use a **custom** window and step size, the user can use the flags **-w** and **-p** as shown in the help section and the screenshots attached. For a detailed understanding of window size and step size, please refer to <https://pmc.ncbi.nlm.nih.gov/articles/PMC2726151/>

To get clean output and remove **intermediate files**, the user can use the **-C (capital C)** flag. **-c (small c)** is for **run_sybr_config.yaml** config file, which will be used for the renamed config file.

This is the output of **example_run.sh** with **Fast track** example dataset:

```
Unlocked working directory.
[INFO] Using custom syntenic parameters from: syntenic_params.yaml
Sybr is Running... e
✓ Completed all chr-FASTA preparations
Sybr is Running... e
✓ Completed chr_size.txt generation (3 entries)
Sybr is Running... e
✓ Completed sorting & filtering of Genus_sps3 at resolution 90000
Sybr is Running... e
✓ Completed sorting & filtering of Genus_sps3 at resolution 30000
✓ Completed sorting & filtering of Genus_sps3 at resolution 60000
Sybr is Running... e
✓ Completed sorting & filtering of Genus_sps2 at resolution 30000
✓ Completed sorting & filtering of Genus_sps2 at resolution 90000
✓ Completed sorting & filtering of Genus_sps2 at resolution 60000
Sybr is Running... e
✓ Completed chainNet processing
Sybr is Running... e
✓ Completed chainNet processing
Sybr is Running... e
✓ Completed netSyntenic processing
Sybr is Running... e
✓ Completed ancestral sequence reconstruction using DESCHRAMBLER
Sybr is Running... e
✓ Completed syntenic tracking of Genus_sps3 in 30000 resolution
Sybr is Running... e
✓ Completed syntenic tracking of Genus_sps3 in 60000 resolution
Sybr is Running... e
✓ Completed syntenic tracking of Genus_sps3 in 90000 resolution
Sybr is Running... e
✓ Completed syntenic tracking of Genus_sps2 in 30000 resolution
Sybr is Running... e
✓ Completed syntenic tracking of Genus_sps2 in 60000 resolution
Sybr is Running... e
✓ Completed syntenic tracking of Genus_sps2 in 90000 resolution
Sybr is Running... e
✓ Completed creating EBA input format to run EBA
Sybr is Running... e
✓ Completed overlap resolution in EBRs
✓ Completed concatenation of non-overlap files
Sybr is Running... e
✓ Completed msHSBs detection in the dataset
✓ Completed splitting EBRs by lineages
Sybr is Running... e
✓ Completed msHSBs FASTA extraction (87 sequences)
Sybr is Running... e
✓ Completed EBRs finding in dataset
✓ Completed syntenic plot generation
Sybr is Running... e
✓ Completed EBRs extraction, 5kb extension, and cleanup
Sybr is Running... e
✓ Completed EBR statistics report
Sybr is Running... e
✓ Completed linear syntenic plot generation
Sybr is Running... e
✓ Completed foreground/background generation for enrichment analysis in msHSBs
[SUCCESS] Pipeline completed successfully in 1041s
[INFO] --clean: removing intermediate files under /home/ajay/Sybr/outputs
[SUCCESS] Cleanup complete - 75 item(s) removed, 0 could not be removed
(sybr) ajay@ajay-SS400TR-34:~/Sybrs
```

(3) Outputs of SyBR:

These are the three folders generated as a result of running SyBR pipeline with all modules activated:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs$ ls
Ancestor_seq_reconstruction  eba_analysis  hgt_overlap  syntenic_processing
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs$
```

The results of the **synteny_processing** module will be generated in the **synteny_processing** folder in the **output folder**. The results of the **eba_analysis** and **enrichment_analysis** modules will be generated inside the **eba_analysis** folder in the output folder. The results of the **chainNet_generation** and **Ancestor_seq_reconstruction** modules will be generated in the **Ancestor_seq_reconstruction** folder within the **output folder**. And the results of the **hgt_overlap_analysis** module will be saved in the **hgt_overlap** output subfolder.

(3.1) synteny_processing:

The **synteny_processing** output folder has two subfolders: **synteny_plots** and **synteny_results**

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/synteny_processing$ ls
chrom_sizes.txt  synteny_plots  synteny_results
```

The **synteny_results** output folder contains subfolders for each **resolution**; within each resolution folder, there are subfolders for **each species**. And in each species folder, there is a **blocks_info** file, which contains the processed synteny information of reference and query at a **particular resolution**. Along with this, **chrom_sizes.txt** file contains chromosome size reference genome.

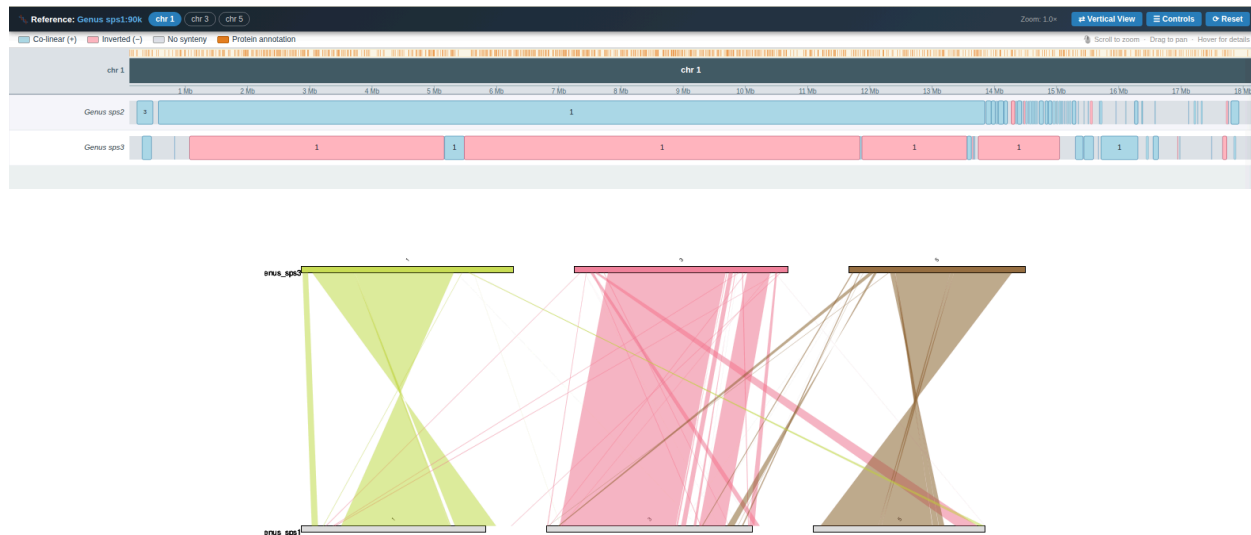
The **blocks_info** file looks like this:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/synteny_processing/synteny_results/30000/Genus_sps2_out$ ls
blocks_info
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/synteny_processing/synteny_results/30000/Genus_sps2_out$ head blocks_info
Reference_genome  Genus_sps1_chromosome  Genus_sps1_start  Genus_sps1_end  Genus_sps2_chromosome  Genus_sps2_start  G
Genus_sps2_end  Sign  Target_genome  HSB  Comments
nominal nominal int  int  nominal int  int  nominal int  nominal
Genus_sps1  1  218387  394750  3  48345  316086  +  Genus_sps2  1
Genus_sps1  1  567677  14209354  1  3974738  16958080  +  Genus_sps2  2
Genus_sps1  1  2502887  2522531  1  6005053  6014061  -  Genus_sps2  3
Genus_sps1  1  3406839  3418223  1  6837172  6848164  -  Genus_sps2  4
Genus_sps1  1  5456654  5571008  1  8795193  8808065  -  Genus_sps2  5
Genus_sps1  1  9723592  9742344  1  12864608  12879342  -  Genus_sps2  6
Genus_sps1  1  13524023  13526953  1  954369  959479  -  Genus_sps2  7
Genus_sps1  1  14271467  14330129  1  17025639  17080319  -  Genus_sps2  8
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/synteny_processing/synteny_results/30000/Genus_sps2_out$
```

The **Synteny_plots** output folder also contains resolution output folders:

```
synteny_plots$ ls
100k  300k  500k
```

In each resolution folder, for each species, two kinds of plots will be generated to visualise synteny results:



(3.2) eba_analysis:

The **eba_analysis** output folder has two subfolders: **EBRs** and **msHSBs**. The EBRs output folder contains all the results related to **evolutionary breakpoint regions**. The msHSBs output folder contains all results related to **multispecies homologous synteny blocks**.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis$ ls
chr_size.txt  EBRs  msHSBs
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis$
```

(3.2.1) EBRs:

This output folder contains a text file (**EBRs_stats.txt**) with statistics for EBRs. This information can be visualized in another HTML file, called **EBRs_stats.html**. A full list of EBRs is available in **EBRs.txt**. The input files for the getENRICH tool for pathway enrichment for all EBRs together and for individual species or nodes are available in folders named after the corresponding species.

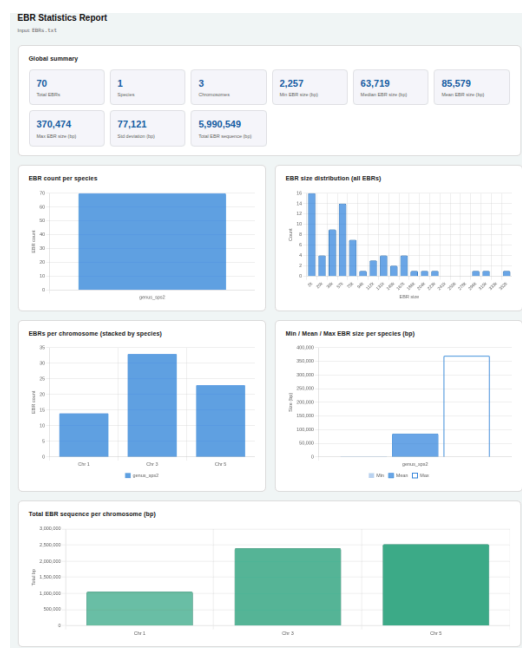
The file structure of EBRs output folder will be like this:

```
EBRs_Enrichment_results EBRs_stats.html EBRs_stats.txt EBRs.txt genus_sps2_lineage
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/EBRs$ tree .
.
├── EBRs_Enrichment_results
│   ├── background.txt
│   ├── EBRs_NCBI_genes_overlap.txt
│   └── foreground.txt
├── EBRs_stats.html
├── EBRs_stats.txt
├── EBRs.txt
└── genus_sps2_lineage
    ├── background.txt
    ├── EBRs_NCBI_genes_overlap.txt
    ├── foreground.txt
    └── genus_sps2.txt
```


This is the format of the main EBR info-containing file:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/EBRs$ ls
EBRs_Enrichment_results  EBRs_stats.html  EBRs_stats.txt  EBRs.txt  genus_sps2_lineage
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/EBRs$ head EBRs.txt
1      15052933      15068404      genus_sps2
1      15437174      15510061      genus_sps2
1      15686561      15700665      genus_sps2
1      15727416      15943940      genus_sps2
1      15943941      15950765      genus_sps2
1      15950766      16115444      genus_sps2
1      16115445      16126009      genus_sps2
1      16126010      16256244      genus_sps2
1      16267714      16297171      genus_sps2
1      16302190      16369499      genus_sps2
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/EBRs$
```

And these are the stats report of EBRs:



```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/EBRs$ ls
EBRs_Enrichment_results  EBRs_stats.html  EBRs_stats.txt  EBRs.txt  genus_sps2_lineage
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/EBRs$ cat EBRs_stats.txt

SYBR Pipeline - EBR Statistics Report
Input: EBRs.txt

GLOBAL SUMMARY
Total EBRs : 70
Total EBR sequence (bp) : 5,990,549
Minimum EBR size : 2,257 bp
Maximum EBR size : 370,474 bp
Mean EBR size : 85,579.3 bp
Median EBR size : 63,719.0 bp
Std deviation : 77,121.0 bp
Chromosomes represented : 3
Species represented : 1

EBR COUNTS AND SIZES BY SPECIES
Species Count %Total Total bp Min bp Mean bp Max bp
genus_sps2 70 100.0% 5,990,549 2,257 85,579.3 370,474

EBR COUNTS AND SIZES BY CHROMOSOME
Chr Count %Total Total bp Min bp Mean bp Max bp
1 14 20.0% 1,056,760 6,824 75,482.9 216,524
3 33 47.1% 2,404,675 2,257 72,868.9 308,273
5 23 32.9% 2,529,114 10,744 109,961.5 370,474

EBR COUNTS PER CHROMOSOME PER SPECIES
Chr genus_sps2 Total
1 14 14
3 33 33
5 23 23
```

(3.2.2) msHSBs:

This output folder contains a list of all msHSB **coordinates** and msHSB **sequences** for the reference genome in **msHSBs.txt** and **msHSB_sequences.fasta**. In addition to these, there are the input files for getENRICH pathway enrichment analysis of msHSBs same as in EBRs.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/msHSBs$ ls
background.txt foreground.txt msHSBs_NCB1_genes_overlap.txt msHSBs_sequences.fasta msHSBs.txt
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/msHSBs$ head msHSBs_sequences.fasta
>1 349518-394750:
TTTCTTATGTTCAACGTTTATTTAGTATATGGATACAAATACAGCCAGCTCGGATGT
ACTCCATCAGAACcatatgaatttcaataatgaaaccAAATCAATATTGATCTACTGATA
TCTGTGCGAAGTGAAGTGAACCGACCATTCACACAACCTTACAACAACCTGATAGAATT
TCGAGTCTGACTATCAACAATTCAAGCTCCTCAACGATACGATAGTACACTCAGACCACGA
CATGACGACGAGGTAGACCATATCACTCTCGACAAGAAGACGCGACGGCTTCGAATGCGAG
CATATCAGTAGTCAAGATATGAACGATCTATTGAACCAATGTCCAATAGAGATCGA
TCACCGCAACACAATGGTAGCATTCAATATCACCAGTTATTGCTGTCGATCGAACAAATCGA
ATGACTTACAGAACACATTTACCAATGATACAGACGAAATTGagcaaaataaactcaact
CAACAGTTCGaaaacgaacaacaacaagaacaatttcGACGTGACGATTGGAA
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/msHSBs$
```

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/msHSBs$ ls
background.txt foreground.txt msHSBs_NCBI_genes_overlap.txt msHSBs_sequences.fasta msHSBs.txt
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/msHSBs$ head msHSBs.txt
1      349518  394750
1      829003  833496
1     1071635 5158990
1     5164340 5305414
1     5309826 5376846
1     5451580 5483876
1     5593102 11027044
1     11031122      11075617
1     11079637      14471826
1     14472996      14494788
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/eba_analysis/msHSBs$
```

(3.3) Ancestor_seq_reconstruction:

The ancestor_seq_reconstruction output folder contained two subfolders: the **data** folder, which contains the **chain** and **net** files used by the **DESHRAMBLER** tool for **ancestor sequence reconstruction**. And **APCFs.300K** contains the final reconstruction results.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_reconstruction$ ls
APCFs.300K data
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_reconstruction$
```

Many files are generated in the **APCFs.300K** directory. Among them, the following files are the most important and useful.

(3.3.1) block_list.txt (in the "SFs" subdirectory)

This file contains the coordinates and identifiers of syntenic fragments (SFs) used in reconstruction.

- Column 1: chromosome or scaffold
- Column 2: start position (0-based); add 1 to obtain the actual start position
- Column 3: end position (1-based)
- Column 4: orientation
- Column 5: identifier

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_reconstruction/APCFs.300K/SFs$ head block_list.txt
chr1      566521 15747356      +      2
chr3     2887155 16277172      +      1
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_reconstruction/APCFs.300K/SFs$
```

(3.3.2) Conserved.Segments (in the "SFs" directory)

This file contains the coordinates of the genomic regions for each species in each SF.

Lines starting with ">": ><identifier of an SF>

Other lines: the coordinates of the genomic regions for each species in the SF. The format is as follows.

<species>.<chromosome or scaffold>:<start position>-<end position> <orientation>

Here, the start position is 0-based, and the end position is 1-based. You need to add 1 to the start position to obtain the actual start position.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_recunstruction/APCFs.300K/SFs$ head Conserved.Segments
>1
sps1.chr3:2887155-16277172 +
sps2.chr3:7111500-19748262 +
sps3.chr3:955059-14099127 +
>2
sps1.chr1:566521-15747356 +
sps2.chr1:3973734-19114999 +
sps3.chr1:3893325-19353589 -
```

(3.3.3) Ancestor.APCF

This file contains the list of ancestral predicted chromosome fragments (APCFs) and the order and orientation of SFs in each APCF.

Line 1: >ANCESTOR <total number of SFs>

Lines starting with "#": # APCF <identifier of APCF>

Other lines: the order and orientation of SFs in the APCF are shown immediately above each line.

SFs are shown using their identifiers (see the block_list.txt file).

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_recunstruction/APCFs.300K$ head Ancestor.APCF
>ANCESTOR      2
# APCF 1
-2 $
# APCF 2
-1 $
```

(3.3.4) Ancestor.ADJS

This file contains the predicted adjacencies of pairs of SFs and their scores used in the Ancestor.APCF file. SFs are shown using their identifiers (see the block_list.txt file).

Column 1: the first SF identifier

Column 2: the second SF identifier

Column 3: adjacency score of the first and the second identifiers

In the first and second columns, 0 means the end of APCFs. For example, "0 64" means the SF 64 is placed at the end of a current APCF.

(3.3.5) APCF_size.txt

This file contains the total length of APCFs.

Column 1: APCF identifier

Column 2: total length

The last line shows the total APCF length.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_reconstruction/APCFs.300K$ head APCF_size.txt
1      15180835
2      13390017
total  28570852
```

(3.3.6) APCFs

This file contains the coordinates of genomic regions for each species that match each APCF.

Lines starting with "#": #<identifier of APCF>

The coordinates of genomic regions of each species (grouped by species) follow this line.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_reconstruction/APCFs.300K$ head APCFs
#1
sps1.chr1:566521-15747356 - [2]
sps2.chr1:3973734-19114999 - [2]
#2
sps1.chr3:2887155-16277172 - [1]
```

(3.3.7) APCF_<SPC>.merged.map

This file contains the mapping of genomic regions of the SPC species for APCFs. Each mapping is shown by using the following three lines.

><identifier of mapping>

APCF.<identifier of APCF>:<start position in APCF>-<end position in APCF> <orientation>

<SPC>.<chromosome or scaffold>:<start position in SPC>-<end position in SPC> <orientation>

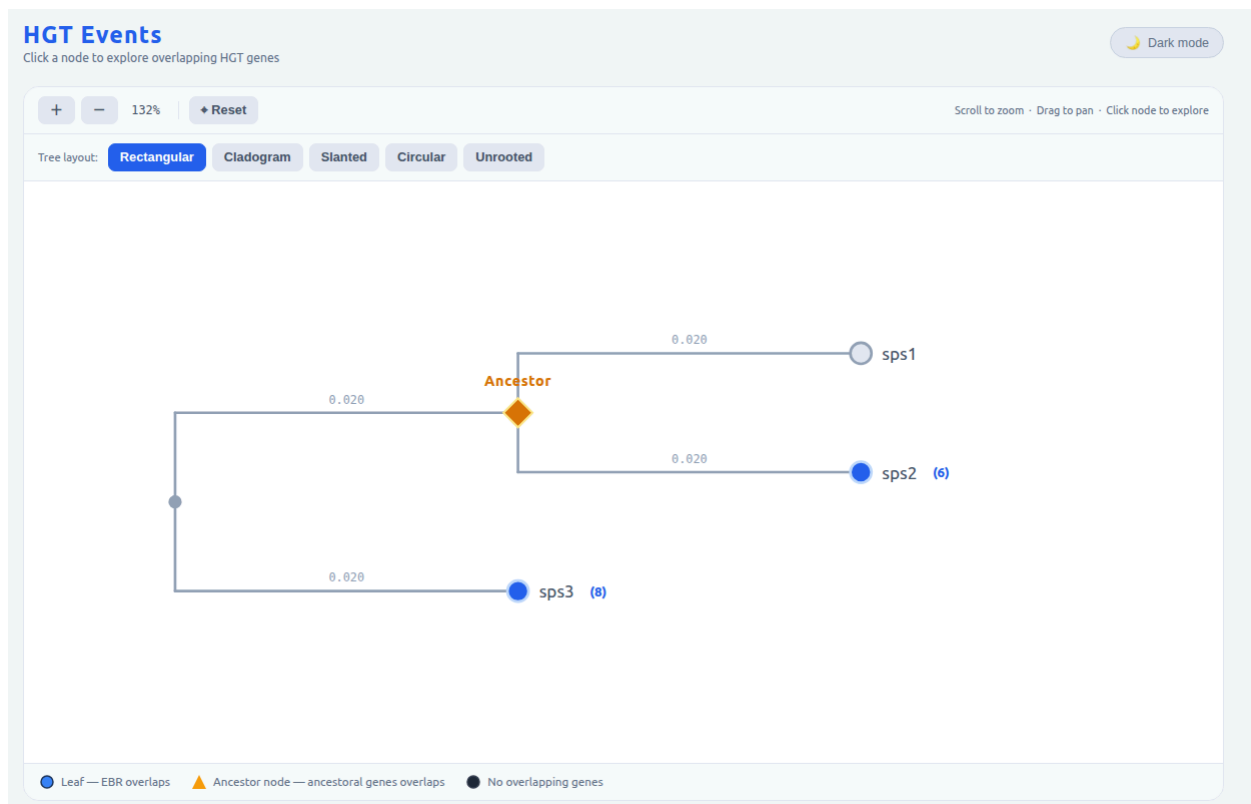
The start position is 0-based, and the end position is 1-based. You need to add 1 to the start position to obtain the actual start position.

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/Ancestor_seq_reconstruction/APCFs.300K$ head APCF_sps1.merged.map
>1
APCF.1:0-15180835 +
sps1.chr1:566521-15747356 -
>2
APCF.2:0-13390017 +
sps1.chr3:2887155-16277172 -
```

(3.4) hgt_overlap:

```
(sybr) ajay@ajay-SS400TR-34:~/Sybr/outputs/hgt_overlap$ ls
hgt_ancestral_genes_overlap_summary.txt  hgt_msHSBs_overlap_summary.txt  hgt_overlapping_EBRs.txt  hgt_tree.html
hgt_EBRs_overlap_summary.txt            hgt_overlapping_ancestral_genes.txt  hgt_overlapping_msHSBs.txt
```

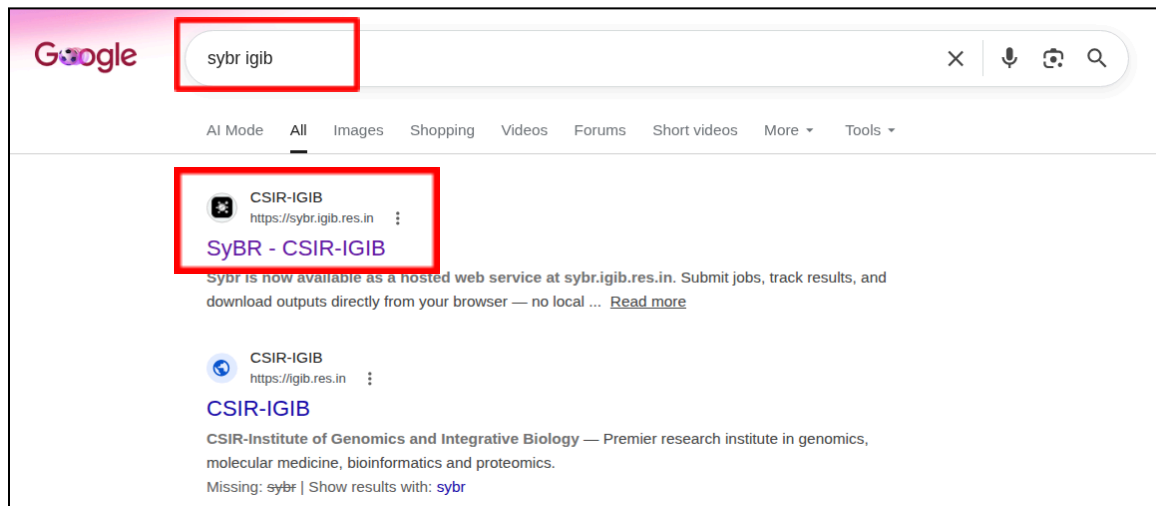
This output folder contains the list of HSBs, msHSBs, EBRs, and ancestral sequences that overlap with the genomic location of HGTs. The information on HGTs is available in three forms. One is the tab-delimited text file; second is the summary file that contains the numbers of HGTs overlapped and paths of used files; and third is a dynamic HTML file that contains the phylogenetic tree with the HGTs information for each node.



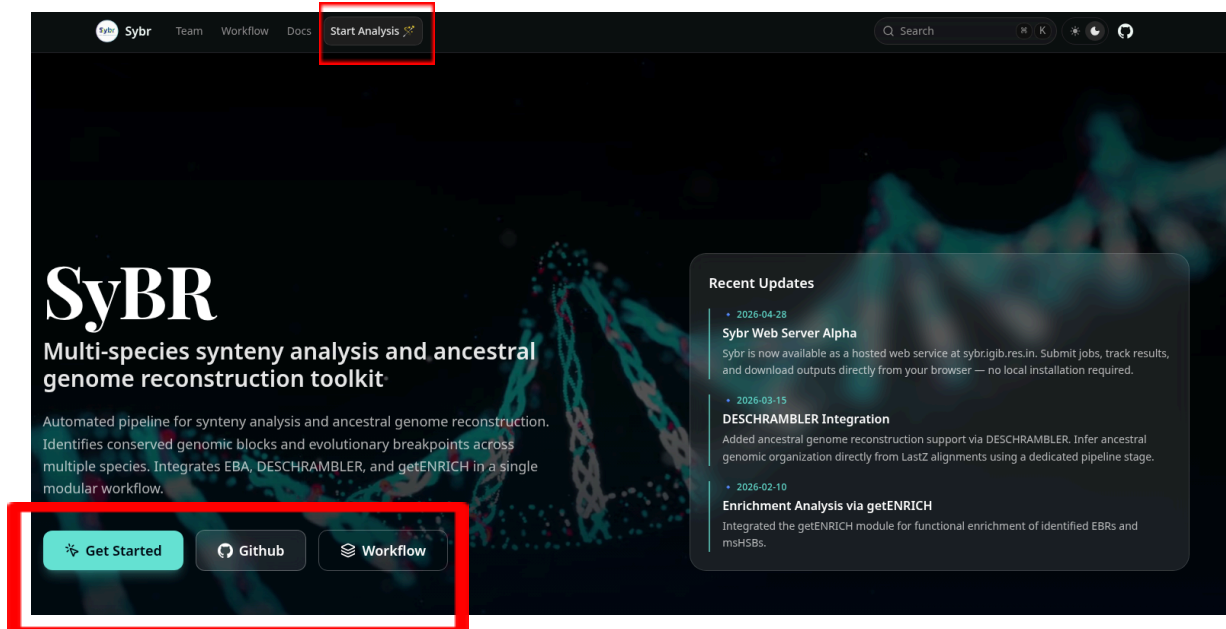
B) Graphical User Interface

(1) Access the SyBR GUI

Search for 'Sybr' or 'Sybr igib' in your favourite web browser, then click on the first link as shown below:



The user will get this landing page of the SyBR website



In this landing page, the user can access three buttons for proceeding. The first button is the link for the SyBR analysis page; the second button is for the source code in GitHub, and the third button is to explore the workflow of this Snakemake pipeline. For further information, the user can scroll down the landing page.

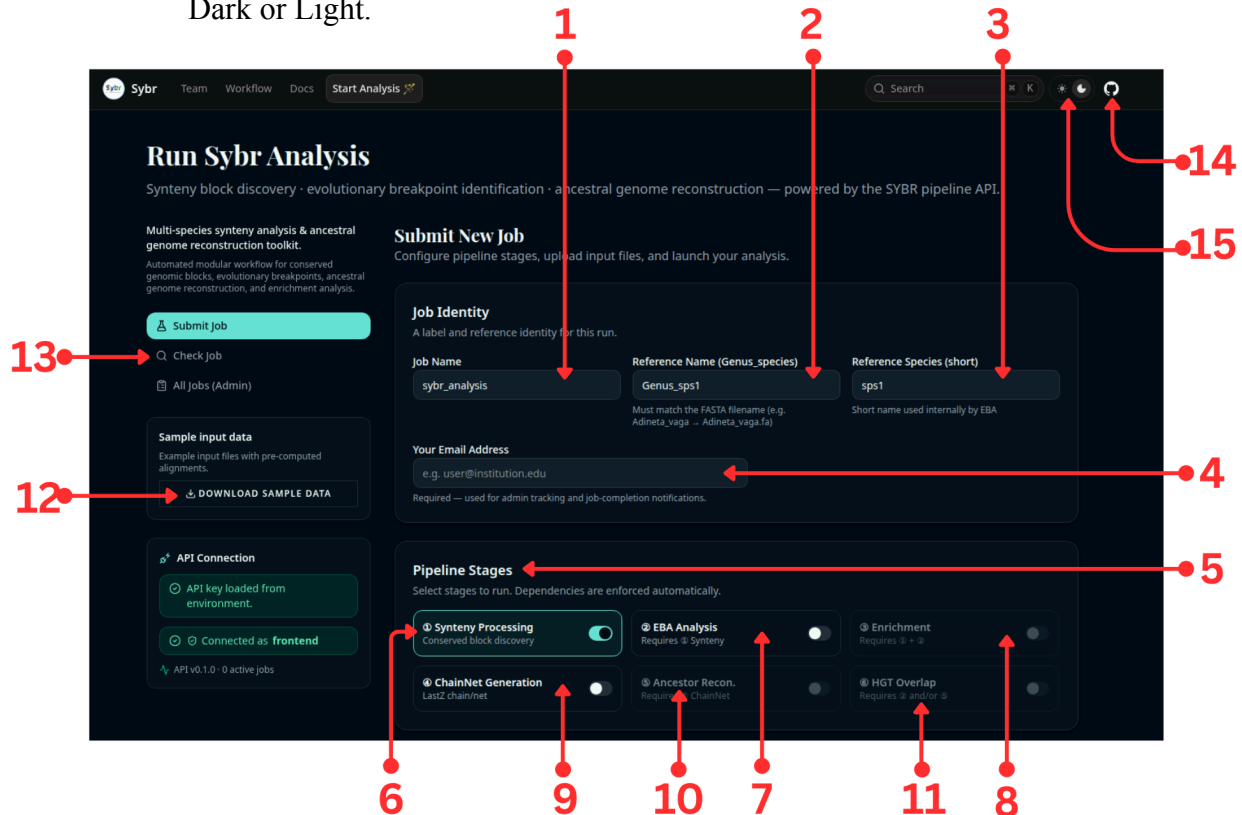
To access the SyBR analysis page, click on '**Get Started**' or '**Start Analysis**'.

(2) Tool Features

After clicking '**Get Started**', the user will get this analysis page. Users need to fill in many fields, select the modules to run and upload the input files before running the SyBR. The details of each element in this page given below:

1. **Job name** : A user-defined name that is used to identify and monitor the provided analysis.
2. **Reference name**: The name (Genus and species: Genus_sps1) of the reference species to be used is to be mentioned here.
3. **Reference species**: The species (sps1) name is to be specified here.
4. **Email address**: The user email id is to be given for the job status updates and notifications.
5. **Pipeline stages**: Select one or more analysis modules to run. The selected modules execute sequentially, with each stage using the output of the previous stage when required.
6. **Syntenic Processing**: Identifies homologous syntenic blocks between genomes through whole-genome alignment and syntenic block assignment.
7. **EBA analysis**: Detects and classifies Evolutionary Breakpoint Regions (EBRs) from syntenic block data. Low-confidence breakpoints are filtered, and the coordinates of retained EBRs can be extended to improve downstream annotation.
8. **Enrichment**: Extracts genes overlapping EBRs and multi-species Homologous Syntenic Blocks (msHSBs) and prepares input files for getENRICH to perform KEGG pathway enrichment analysis.
9. **Chain NET Generation**: Uses UCSC genome utilities to generate chain and net files from LASTZ alignments, producing hierarchical genome alignments and syntenic relationships.
10. **Ancestor Recons**: Reconstructs ancestral genome organization using DESCHRAMBLER, chain/net files, whole-genome alignments, and a Newick species tree.
11. **HGT Overlap**: Maps horizontally transferred genes (HGTs) onto EBRs, msHSBs, and reconstructed ancestral genomes using the provided HGT annotation file.

12. **Download data:** Download sample data to test and understand the tool workflow.
13. **Check job:** Monitor the current status of submitted jobs, including queued, running, completed, or failed analyses.
14. **GitHub link:** Provides access to the SyBR source code, documentation, installation instructions, and issue reporting repository.
15. **Mode selection:** Allows users to choose the appropriate mode of visualization: Dark or Light.

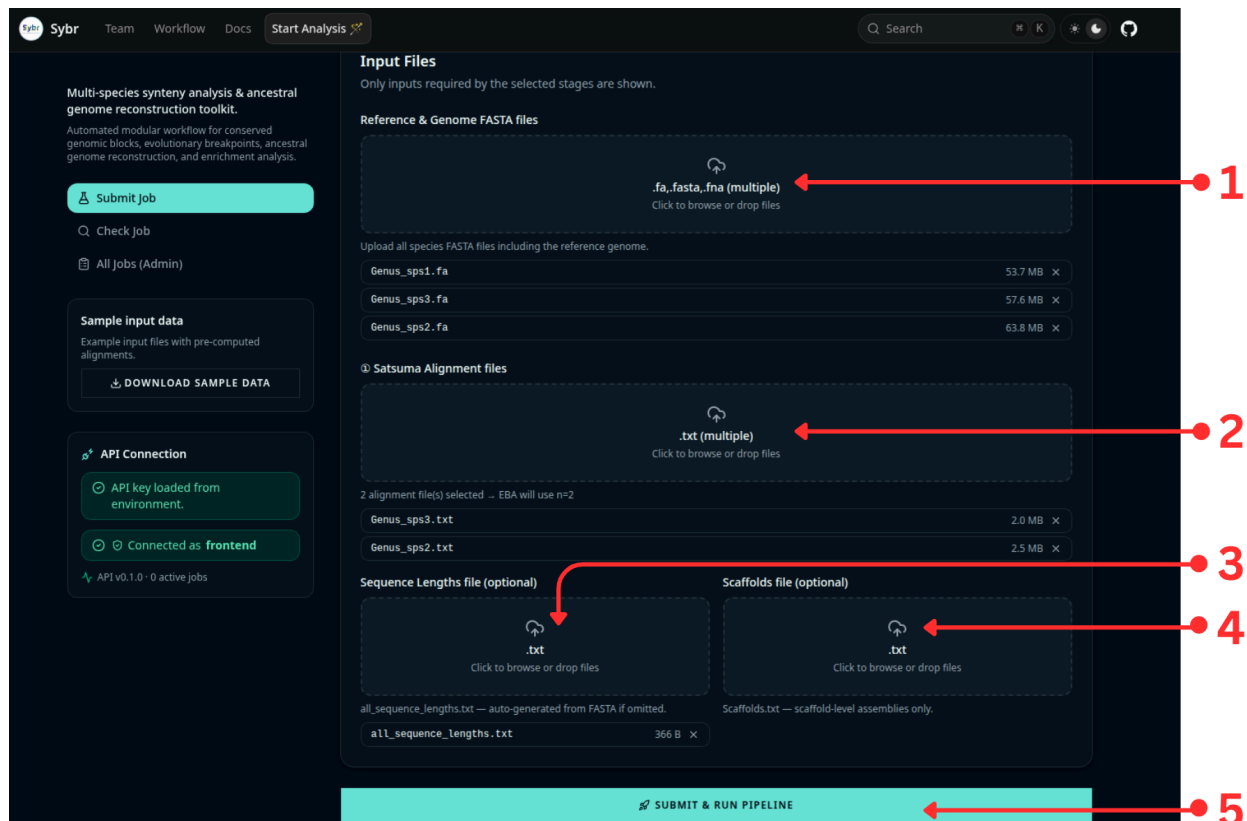


(3) Upload the input files

Scroll down the page to upload the required input files for the analysis. Users can upload genome FASTA files, alignment files, and optional supporting files.

Follow the steps given below to run the analysis:

1. **Reference & Genome FASTA files** : Upload one or more genome sequence files in **.fa**, **.fasta**, or **.fna** format.
2. **Satsuma Alignment files** : Upload whole-genome alignment files generated by Satsuma in **.txt** format.
3. **Sequence Lengths File (Optional)**: Upload **all_sequence_lengths.txt** in **.txt** format. If this file is not provided, the pipeline automatically generates it from the uploaded FASTA files.
4. **Scaffolds File (Optional)**: Upload **scaffolds.txt** in **.txt** format for scaffold-level genome assemblies. This file is not required for chromosome-level assemblies.
5. **Submit & Run Pipeline**: After uploading all required input files, click this button to start the selected pipeline analysis.

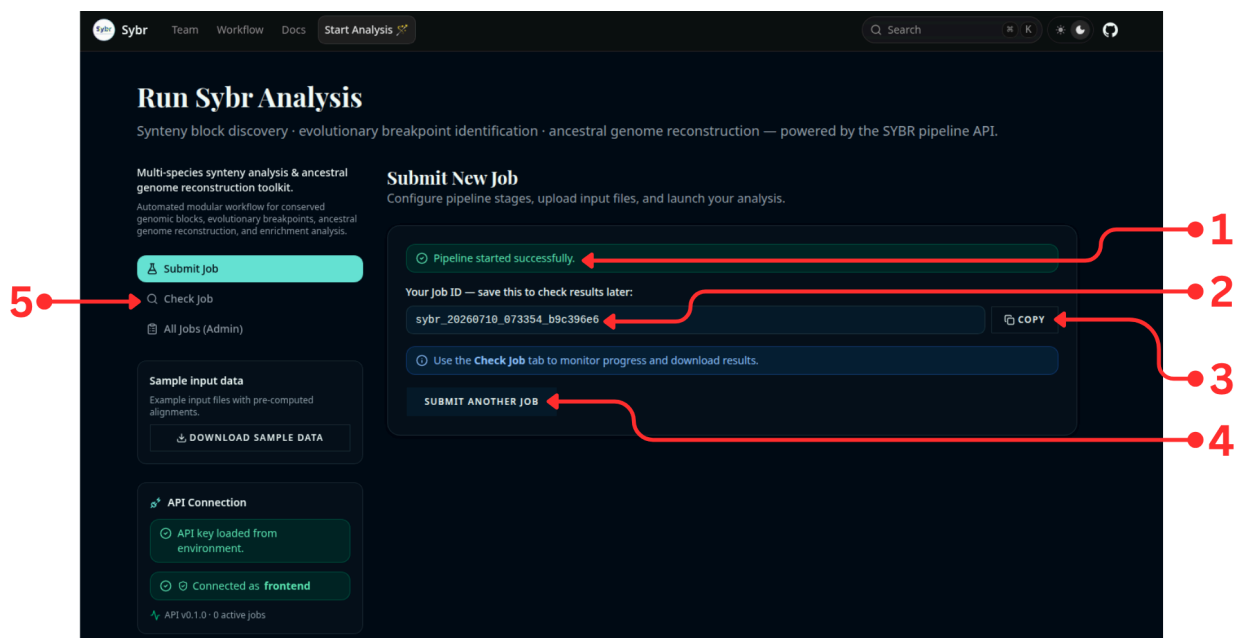


(4) Job submission

After the job is submitted, the following page appears.

1. **Pipeline Started Successfully**: After job submission, a confirmation message stating "**Pipeline started successfully**" is displayed.

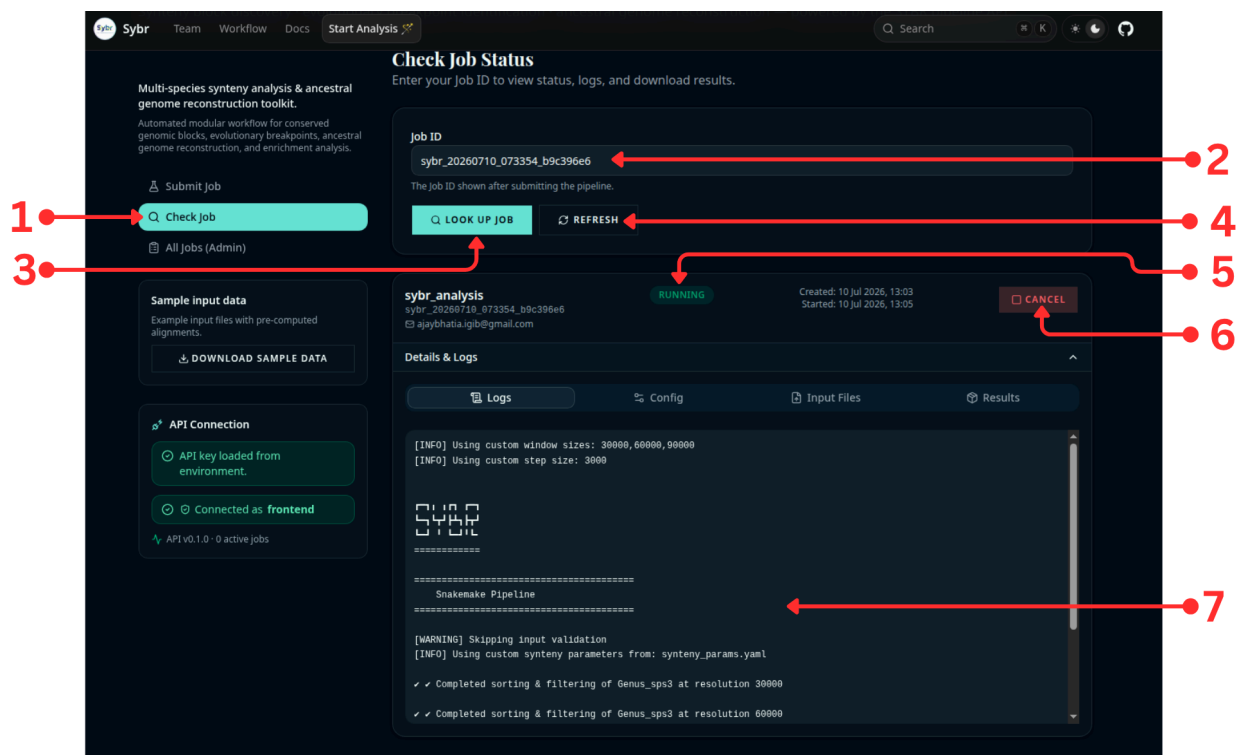
2. **Unique Job ID:** A unique Job ID is automatically generated for each submitted analysis.
3. **Copy Job ID:** Use the **Copy** button to copy the Job ID. Save this ID for future reference, as it is required to track the analysis.
4. **Submit Another Job:** Click this option to start a new analysis by submitting another job.
5. **Check Job:** Click this option to monitor the status of a previously submitted job using the recorded Job ID.



(4) Checking Job status

The progress of a submitted job can be monitored using the “**Check job**” button option the left sidebar.

1. **Check Job:** Allows users to check the status of a previously submitted job using its unique Job ID.
2. **Job ID:** Enter the Job ID of the analysis whose status you want to track.
3. **Look Up Job:** Click this button to retrieve the details and current status of the entered Job ID.
4. **Refresh:** Updates the displayed job status and pipeline progress with the latest information.
5. **Running:** Indicates that the submitted job is currently being executed by the pipeline.
6. **Cancel:** Click this button to terminate a running job.
7. **Logs:** Displays the execution logs of the pipeline. Any errors, warnings, or progress updates generated during the analysis are recorded here.



(4) Results

After the completion of the submitted job, the user can access the results.

1. **Completed:** This message indicates the completion of the running job.
2. **Delete:** If the user wants to remove the completed job, the user can use the delete button.

3. **Results:** Click the “**Results**” tab to view and download the output files generated by the completed analysis.

The screenshot displays the Sybr web application interface. The top navigation bar includes links for Team, Workflow, Docs, and Start Analysis. A search bar is located on the right. The main content area is titled "Check Job Status" and prompts the user to enter a Job ID. A sidebar on the left contains options like "Submit Job", "Check Job", and "All Jobs (Admin)". The central panel shows a job named "sybr_analysis" with a status of "COMPLETED" and a progress bar at 100%. Below this, there are tabs for "Logs", "Config", "Input Files", and "Results". The "Results" tab is highlighted with a red arrow labeled "3". A red arrow labeled "1" points to the "Job ID" input field, and a red arrow labeled "2" points to the "DELETE" button.

Check Job Status
Enter your Job ID to view status, logs, and download results.

Job ID: sybr_20260710_073354_b9c396e6
The Job ID shown after submitting the pipeline.

LOOK UP JOB REFRESH

sybr_analysis
sybr_20260710_073354_b9c396e6
ajaybhatia.igib@gmail.com

COMPLETED 100%

Created: 10 Jul 2026, 13:03
Started: 10 Jul 2026, 13:05
Done: 10 Jul 2026, 13:28

DELETE

Details & Logs

Logs Config Input Files Results

- ✓ Completed synteny tracking of Genus_sps2 in 60000 resolution
- ✓ Completed synteny tracking of Genus_sps2 in 90000 resolution
- ✓ Completed creating EBA input format to run EBA
- ✓ Completed overlap resolution in EBRs
- ✓ Completed concatenation of non-overlap files
- ✓ Completed splitting EBRs by lineages
- ✓ Completed synteny plot generation
- ✓ Completed linear synteny plot generation

[SUCCESS] Pipeline completed successfully in 1379s

(5) Download Results

Users can download the computed results after the job completion.

1. **Results:** Click on the Results tab to access the links to download the results.

2. **DOWNLOAD ALL (zip):** To download the complete compressed result file, click on the DOWNLOAD ALL (zip) button. This will lead to saving the zipped file in the user's download folder.
3. **Selective Downloads:** The user can download the specific result files or folders instead of the complete zipped file using a tick mark on output files/folders.
4. **DOWNLOAD SELECTED:** The selected output file/folders can be downloaded by clicking on the **DOWNLOAD SELECTED** button. This will download the individual files separately.

The screenshot displays the Sybr web application interface. The top navigation bar includes 'Sybr', 'Team', 'Workflow', 'Docs', and 'Start Analysis'. The main heading is 'Run Sybr Analysis', with a subtitle 'Syntenic block discovery · evolutionary breakpoint identification · ancestral genome reconstruction — powered by the SYBR pipeline API.' Below this, there's a 'Check Job Status' section where a job ID is entered: 'sybr_20260710_073354_b9c396e6'. The job status is 'COMPLETED' with a progress bar at 100%. The 'Details & Logs' section shows tabs for 'Logs', 'Config', 'Input Files', and 'Results'. Under 'Results', there's a 'DOWNLOAD ALL (ZIP)' button. Below that, a list of outputs is shown, with 'syntenic_processing' selected (indicated by a red arrow and the number 3). A 'DOWNLOAD SELECTED' button is also present (indicated by a red arrow and the number 4). The interface is dark-themed with teal accents.

1. Results tab

2. DOWNLOAD ALL (ZIP) button

3. syntenic_processing folder

4. DOWNLOAD SELECTED button